

SOUGATA SAHA
IT-A2-037 OOP ASSIGNMENT 4

ASSIGNMENT 4

Q36. Write a class “Point” which stores coordinates in (x, y) form. Define necessary constructor, destructor and other reader/writer functions. Now overload ‘-’ operator to calculate the distance between two points.

```
#include <iostream>
#include <cmath>
using namespace std;
class Point {
private:
    double x, y;
public:
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    ~Point() {}
    double operator-( Point& p) {
        return sqrt(pow(x - p.x, 2) + pow(y - p.y, 2));
    }
};
```

```

int main() {
    Point p1(3, 4), p2(0, 0);
    cout << "Distance: " << p1 - p2 << endl;
    return 0;
}

```

Q36. Design a class Complex that includes all the necessary functions and operators like =, +, -, *, /

```

#include <iostream>
using namespace std;

class Complex {
private:
    double real, imag;
public:
    Complex(double r = 0, double i = 0) : real(r), imag(i) {}
    Complex& operator=(Complex& c) {
        real = c.real;
        imag = c.imag;
        return *this;
    }
    Complex operator+(Complex& c) {
        return Complex(real + c.real, imag + c.imag);
    }
    Complex operator-(Complex& c) {
        return Complex(real - c.real, imag - c.imag);
    }
    Complex operator*(Complex& c) {
        return Complex(real * c.real - imag * c.imag, real * c.imag + imag * c.real);
    }
}

```

```

Complex operator/(Complex& c) {
    double denominator = c.real * c.real + c.imag * c.imag;
    double realPart = (real * c.real + imag * c.imag) / denominator;
    double imagPart = (imag * c.real - real * c.imag) / denominator;
    return Complex(realPart, imagPart);
}

void display() {
    cout << real << " + " << imag << "i" << endl;
}
};

int main() {
    Complex c1(3, 4), c2(1, 2);
    Complex c3 = c1 + c2;
    Complex c4 = c1 - c2;
    Complex c5 = c1 * c2;
    Complex c6 = c1 / c2;

    c3.display();
    c4.display();
    c5.display();
    c6.display();

    return 0;
}

```

Q38. Implement a class “Quadratic” that represents second-degree polynomial i.e. polynomial of type ax^2+bx+c . The class will require three data members corresponding to a, b and c. Implement the following: a. A constructor (including a default constructor which create a null polynomial) b. Overload the addition operator to add two polynomials of degree 2. c. Overload $<<$ and $>>$ operators to print and read polynomials. d. A function to compute the value of polynomial for a given x. e. A function to compute roots of the equation $ax^2+bx+c=0$. Remember, root may be a complex number. You may implement “Complex” class to represent root of the quadratic equation.

```
#include <iostream>
#include <cmath>
using namespace std;

class Complex {
private:
    double real, imag;

public:
    Complex(double r = 0, double i = 0) : real(r), imag(i) {}
    void display() { cout << real << " + " << imag << "i" << endl; }
};

class Quadratic {
private:
    double a, b, c;

public:
    Quadratic(double a = 0, double b = 0, double c = 0) : a(a), b(b), c(c) {}

    Quadratic operator+(Quadratic& q) {
        return Quadratic(a + q.a, b + q.b, c + q.c);
    }

    friend istream& operator>>(istream& in, Quadratic& q) {
        in >> q.a >> q.b >> q.c;
        return in;
    }

    friend ostream& operator<<(ostream& out, Quadratic& q) {
        out << q.a << "x^2 + " << q.b << "x + " << q.c;
```

```

        return out;
    }

double compute(double x) {
    return a * x * x + b * x + c;
}

void computeRoots() {
    double discriminant = b * b - 4 * a * c;
    if (discriminant >= 0) {
        double root1 = (-b + sqrt(discriminant)) / (2 * a);
        double root2 = (-b - sqrt(discriminant)) / (2 * a);
        cout << "Roots: " << root1 << " and " << root2 << endl;
    } else {
        Complex root1(-b / (2 * a), sqrt(-discriminant) / (2 * a));
        Complex root2(-b / (2 * a), -sqrt(-discriminant) / (2 * a));
        cout << "Roots: ";
        root1.display();
        root2.display();
    }
}

};

int main() {
    Quadratic q1(1, -3, 2), q2(2, 1, 3);
    Quadratic q3 = q1 + q2;

    cout << "q1: " << q1 << endl;
    cout << "q2: " << q2 << endl;
    cout << "q3: " << q3 << endl;

    cout << "Value of q1 for x=2: " << q1.compute(2) << endl;
}

```

```
cout << "Roots of q1: ";  
q1.computeRoots();  
  
return 0;  
}
```

Q39. . A program is given as follows: class INT { int i; public : INT(int a):i(a){ ~INT() {} }; int main() { int x = 3; INT y = x; y++ = ++y; x = y; return 0; }

```
#include <iostream>  
using namespace std;
```

```
class INT {  
    int i;
```

```
public:
```

```
    INT(int a) : i(a) {}  
    ~INT() {}
```

```
    INT& operator++() {  
        ++i;  
        return *this;  
    }
```

```
    INT operator++(int) {  
        INT temp = *this;  
        i++;  
        return temp;  
    }
```

```
    operator int() {
```

```

        return i;
    }
    void display(){
        cout<<i<<endl;
    }
};

```

```

int main() {
    int x = 3;
    INT y = x;
    y++ = ++y;
    x = y;
    y.display();
}

```

Q40. Design and implement class(es) to support the following main program. int main() { IntArray i(10); for(int k = 0; k < 10; k++) i[k] = k; cout << i; return 0; }

```

#include<iostream>
using namespace std;
class Array{
    int *p;
    int size;
public:
    Array(int s){
        size=s;
        p=new int[size];
    }
    int& operator [] ( int index){
        return p[index];
    }
    friend ostream& operator <<(ostream &os,Array ob){
        for(int i=0;i<ob.size;i++){
            os<<ob.p[i]<<endl;
        }
    }
}

```



```

    }
};

int main(){
    Array i(10);
    for(int k = 0; k < 10; k++)
        i[k] = k;
    cout<<i;
}

```

Q41. You are given a main program: `int main() { Integer a = 4, b = a, c; c = a+b++; int i = a; cout << a << b << c; return 0; }` Design and implement class(es) to support the main program.

```

#include<iostream>

using namespace std;

class integer{
    int a;
public:
    integer(int x=0) :a(x){}
    ~integer() {}
    operator int(){
        return a;
    }

    integer operator++(int){
        integer temp;
        temp=a++;
        return temp;
    }

    friend ostream& operator <<(ostream &os,integer ob){
        os<<ob.a<<endl;
    }
};

int main(){
    integer a=4,b=a,c;
    c=a+b++;
}

```

```

    int i=a;

    cout<<a<<b<<c;

    return 0;

}

```

Q42. 42. Design and implement class(es) to support the following code segment. Table t(4, 5), t1(4, 5); cin >> t; t[0][0] = 5; int x = t[2][3]; t1 = t; cout << t << “\n” << t1;

```

#include <iostream>

using namespace std;

```

```

class Table {
private:
    int rows, cols;
    int** data;

public:
    Table(int r, int c) : rows(r), cols(c) {
        data = new int*[rows];
        for (int i = 0; i < rows; ++i)
            data[i] = new int[cols]();
    }

```

```

    ~Table() {
        for (int i = 0; i < rows; ++i)
            delete[] data[i];
        delete[] data;
    }

```

```

    Table(const Table& t) : rows(t.rows), cols(t.cols) {
        data = new int*[rows];
        for (int i = 0; i < rows; ++i) {
            data[i] = new int[cols];
            for (int j = 0; j < cols; ++j)

```

```
        data[i][j] = t.data[i][j];
    }
}
```

```
Table& operator=(const Table& t) {
    if (this != &t) {
        for (int i = 0; i < rows; ++i)
            delete[] data[i];
        delete[] data;

        rows = t.rows;
        cols = t.cols;

        data = new int*[rows];
        for (int i = 0; i < rows; ++i) {
            data[i] = new int[cols];
            for (int j = 0; j < cols; ++j)
                data[i][j] = t.data[i][j];
        }
    }
    return *this;
}
```

```
int* operator[](int index) {
    return data[index];
}
```

```
friend istream& operator>>(istream& in, Table& t) {
    for (int i = 0; i < t.rows; ++i)
        for (int j = 0; j < t.cols; ++j)
            in >> t.data[i][j];
    return in;
}
```

```

    }

    friend ostream& operator<<(ostream& out, const Table& t) {
        for (int i = 0; i < t.rows; ++i) {
            for (int j = 0; j < t.cols; ++j)
                out << t.data[i][j] << " ";
            out << endl;
        }
        return out;
    }
};

```

```

int main() {
    Table t(4, 5), t1(4, 5);
    cin >> t;
    t[0][0] = 5;
    int x = t[2][3];
    t1 = t;
    cout << t << "\n" << t1;
    return 0;
}

```

Q43. Design and implement class(es) to support the following code segment. Index in(4), out(10); int x = in; int y = in + out; in = 2; Integer i; i = in;

```

#include <iostream>
using namespace std;

class Index {
    int value;

public:
    Index(int v = 0) : value(v) {}
}

```

```
operator int() {  
    return value;  
}
```

```
Index& operator=(int v) {  
    value = v;  
    return *this;  
}
```

```
Index operator+(Index& other) {  
    return Index(value + other.value);  
}
```

```
int get_index(){  
    return value;  
}  
  
void display(){  
    cout<<value<<endl;  
}  
};
```

```
class Integer {  
    int value;
```

```
public:
```

```
    Integer(int v = 0) : value(v) {}
```

```
    void operator=(Index& in) {  
        value=in.get_index();
```

```
    }
```

```
    void display(){
```

```
        cout<<value<<endl;
    }
};
```

```
int main() {
    Index in(4), out(10);
    int x = in;
    int y = in + out;
    in = 2;
    Integer i;
    i = in;
    i.display();
    in.display();
}
```

